

Введение

1. Алгоритм. Свойства алгоритма. Способы описания алгоритма

Если нужно написать программу на каком-либо языке программирования, то с начала следует составить алгоритм решения задачи.

Алгоритм — это точное и простое описание последовательности действий для решения данной задачи. Алгоритм содержит несколько шагов, которые должны выполняться в определенной последовательности. Каждый шаг алгоритма может состоять из одной или нескольких простых операций.

Для алгоритма важен не только набор действий, но и то, в каком порядке они выполняются. Понятие алгоритма в информатике является фундаментальным.

Таким же, какими являются понятия точки, прямой и плоскости в геометрии, вещества в химии, пространства и времени в физике и т. д.

Свойства алгоритма:

- дискретность (прерывность, раздельность) алгоритм должен представлять процесс решения задачи как последовательное выполнение простых шагов (этапов);
- определенность каждый шаг алгоритма должен быть четким и однозначным. Выполнение алгоритма носит механический характер и не требует никаких дополнительных сведений о решаемой задаче;
- результативность - алгоритм должен приводить к решению задачи за конечное число шагов;
- массовость алгоритм решения разрабатывается в общем виде. т. е. он должен быть применим для решения некоторого класса задач, различающихся лишь исходными данными.

Способы описания алгоритмов:

- словесный;
- графический;
- табличный;
- формульный.

Словесный способ каждый использует ежедневно, пересказывая собеседнику, например, различные инструкции, правила, кулинарные рецепты. т. е. какую-то последовательность, приводящую к конечному результату.

Графический способ представления алгоритмов является более компактным и наглядным по сравнению со словесным. Как часто для

лучшего понимания той или иной ситуации проще начертить какую-то схему, план, согласно которым необходимо будем действовать. В программировании данный способ предпочтителен, поскольку позволяет с помощью последовательности функциональных блоков, каждый из которых соответствует выполнению одного или нескольких действий, представить ход решения той или иной задачи. Такое представление алгоритма называется структурной схемой алгоритма, или блок-схемой.

Табличный способ используется, например, в бухгалтерии при составлении ежегодных отчетов, сводок и т. д.

Формульный способ находит свое применение при решении задач из области математики, физики и т. д. Например, при решении квадратного уравнения приступают к нахождению дискриминанта уравнения, а затем, в зависимости от полученного результата, находят корни уравнения по известным всем формулам.

2. Назначение функциональных блоков

При отрисовке блок-схем используются функциональные блоки, представленные на рисунке 1.

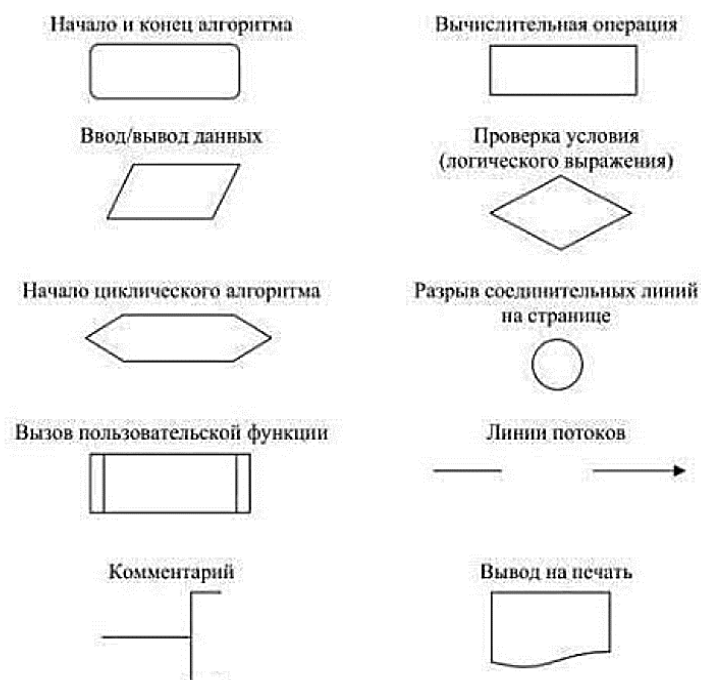


Рисунок 1 – Функциональные блоки при отрисовке блок-схем

3. Основные этапы решения задач

Процесс решения некоторой задачи в среде программирования, как правило, состоит из нескольких этапов, часть из которых выполняется пользователем, а часть компьютером.

1- й этап. Общая постановка задачи.

На этом этапе описывается содержание задачи, составляется перечень исходных данных.

2-й этап. Разработка математической модели.

Цель этого этапа состоит в установлении формализованных связей между исходными данными и искомыми результатами. Этап заключается в записи расчетных формул или функциональных зависимостей.

3-й этап. Разработка алгоритма.

Этап состоит в описании последовательности действий, в результате которых может быть получено решение задачи.

4-й этап. Разработка программы.

Программа составляется в полном соответствии с разработанным алгоритмом решения задачи.

5-й этап. Отладка программы.

Процесс поиска ошибок в программе и их устранение.

6-й этап. Анализ результатов.

Позволяет принять решение о необходимости внесения изменений в программу, проведении дополнительных расчетов или их окончании.

4. Алфавит языка Python

Изучение любого языка начинается с изучения алфавита, из букв складываются слова, из слов предложения. То же происходит и при изучении языка программирования. Сначала необходимо уяснить, какие символы можно использовать для записи слов языка, из которых можно формировать определенные конструкции. Итак, в алфавит языка Python входят латинские буквы от a до z и от A до Z.

В Python есть различия между прописными и строчными буквами алфавита, например, `chislo`, `CHISLO`, `Chislo` разные имена переменных.

- цифры от 0 до 9;
- специальные символы, например `+`, `-`, `*`, `/`;
- зарезервированные (служебные) слова: `for`, `if`, `class`, `def` и т. д.

5. Комментарии в языке Python

Комментарии предназначены для вставки пояснений в текст программы, интерпретатор полностью их игнорирует. Внутри комментария может располагаться любой текст, включая инструкции, которые выполнять не следует.

В языке **Python** присутствует только *однострочный комментарий*. Он начинается с символа `#`:

Это комментарий

Однострочный комментарий может начинаться не только с начала строки, но и располагаться после инструкции. Например, в следующем примере комментарий расположен после инструкции, предписывающей вывести надпись "Привет, мир!":

```
print("Привет, мир!") # Выводим надпись с помощью функции print()
```

Если же символ комментария разместить перед инструкцией, то она не будет выполнена:

```
# print("Привет, мир!") Эта инструкция выполнена не будет
```

Если символ # расположен внутри кавычек или апострофов, то он не является символом комментария:

```
print("# Это НЕ комментарий")
```

Так как в языке Python нет многострочного комментария, то часто комментируемый фрагмент размещают внутри утроенных кавычек (или утроенных апострофов):

```
"""
Эта инструкция выполнена не будет
print ("Привет, мир!")
"""
```

Следует заметить, что этот фрагмент кода не игнорируется интерпретатором, т. к. он не является комментарием. В результате выполнения фрагмента будет создан объект строкового типа. Тем не менее инструкции внутри утроенных кавычек выполнены не будут, поскольку интерпретатор сочтет их простым текстом. Такие строки являются строками документирования, а не комментариями.

6. Ввод данных в языке Python

Для ввода данных в Python 3 предназначена функция **input ()**, которая получает данные со стандартного ввода **stdin**. Функция имеет следующий формат:

```
[<Значение> = ] input([<Сообщение>])
```

Для примера переделаем первую программу так, чтобы она здоровалась не со всем миром, а только с нами (рисунок 2).

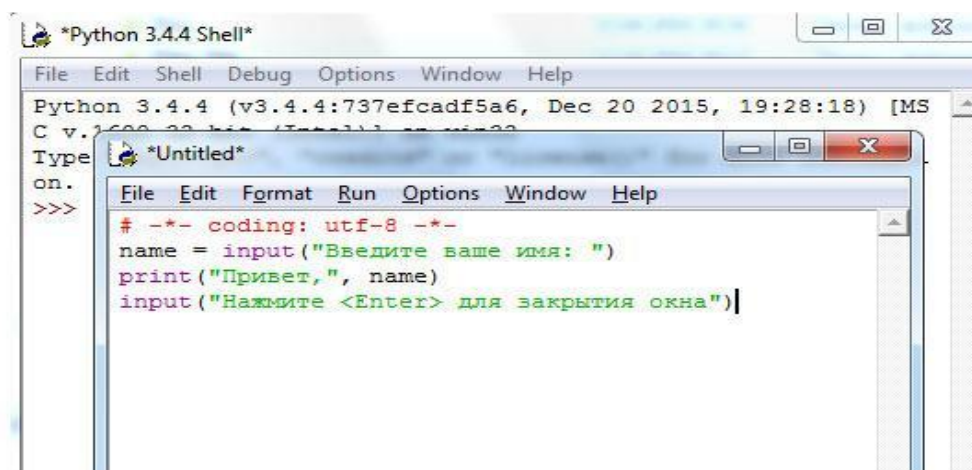


Рисунок 2 – Пример использования функции `input ()`

Чтобы окно сразу не закрылось, повторно вызываем функцию **`input ()`**. В этом случае окно не закроется, пока не будет нажата клавиша **Enter**.

Функция **`input ()`** позволяет вводить выражения, которые затем, с помощью функции **`eval ()`**, можно выполнить. Например, следующая программа запрашивает выражение и вычисляет его (рисунок 3):

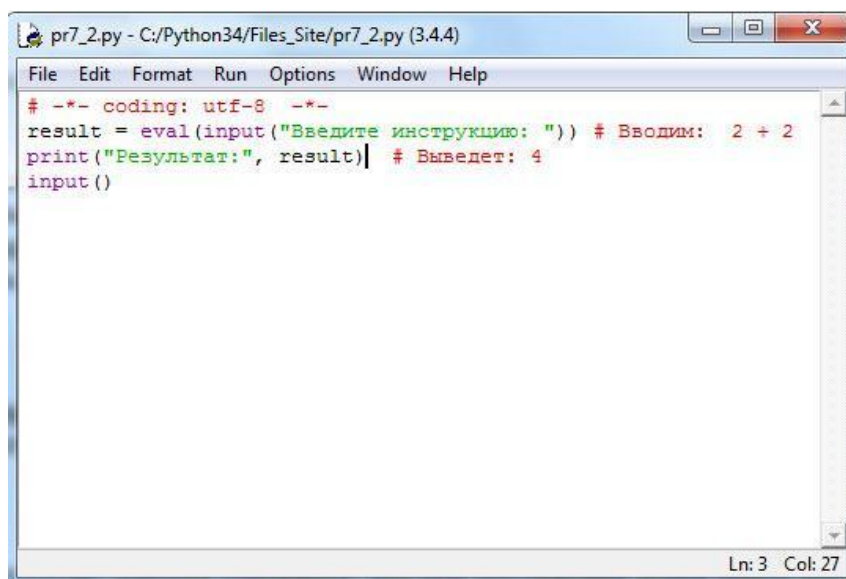


Рисунок 3 – Использование функции `eval ()`

7. Вывод результатов работы программы в языке Python

Вывести результаты работы программы можно с помощью функции **`print ()`**. Функция имеет следующий формат:

`print([<Объекты>][, sep=' '][, end='\n'][, file=sys.stdout][, flush=False])`

Функция **print ()** преобразует объект в строку и посылает ее в стандартный вывод **stdout**. С помощью параметра **file** можно перенаправить вывод в другое место - например, в файл. При этом, если параметр **flush** имеет значение **False**, выводимые значения будут принудительно записаны в файл. После вывода строки автоматически добавляется символ перевода строки:

```
print ("Строка 1")  
print ("Строка 2")
```

Результат:

```
Строка 1  
Строка 2
```

Если необходимо вывести результат на той же строке, то в функции **print ()** данные указываются через запятую:

```
print ("Строка 1", "Строка 2")
```

Результат:

```
Строка 1 Строка 2
```

Как видно из примера, между выводимыми строками автоматически вставляется пробел. С помощью параметра **sep** можно указать другой символ. Например, выведем строки без пробела между ними:

```
print ("Строка 1", "Строка 2", sep="")
```

Результат:

```
Строка 1Строка 2
```

После вывода объектов в конце добавляется символ перевода строки. Если необходимо произвести дальнейший вывод на той же строке, то в параметре **end** следует указать другой символ:

```
print ("Строка 1", "Строка 2", end=" ")  
print ("Строка 3")
```

Выведет: Строка 1 Строка 2 Строка 3

Для вывода результатов работы программы вместо функции **print ()** можно использовать метод **write ()** объекта **sys.stdout**:

```
import sys # Подключаем модуль sys  
sys.stdout.write ("Строка") # Выводим строку
```